

# Тема 2.5

## Функции

# Зачем нужны процедуры?

```
print ( "Ошибка программы" )
```

много раз!

Процедура:

*define*  
определить

```
def Error() :  
    print( "Ошибка программы" )
```

```
n = int ( input() )  
if n < 0:  
    Error()
```

ВЫЗОВ  
процедуры

# Что такое процедура?

**Процедура** — вспомогательный алгоритм, который выполняет некоторые действия.

- текст (расшифровка) процедуры записывается **до** её вызова в основной программе
- в программе может быть **много процедур**
- чтобы процедура заработала, нужно **вызвать** её по имени из основной программы или из другой процедуры

# Процедура с параметрами

Задача. Вывести на экран запись целого числа (0..255) в 8-битном двоичном коде.

много раз!

## Алгоритм:

$$178 \Rightarrow 10110010_2$$



Как вывести первую цифру?

$n :=$ 

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |

 $_2$  разряды

$n \ // \ 128$

$n \ \% \ 128$



Как вывести вторую цифру?

$n1 \ // \ 64$

# Процедура с параметрами

**Задача.** Вывести на экран запись целого числа (0..255) в 8-битном двоичном коде.

## Решение:

```
k = 128
while k > 0:
    print ( n // k,
            end = " " )
    n = n % k
    k = k // 2
```

**178  $\Rightarrow$  10110010**



# Результат зависит от $n$ !

[illegible]

# Процедура с параметрами

**Параметры** – данные, изменяющие работу процедуры.

локальная  
переменная

```
def printBin( n ) :  
    k = 128  
    while k > 0 :  
        print ( n // k, end = "" )  
        n = n % k;  
        k = k // 2
```

```
printBin ( 99 )
```

значение параметра  
(**аргумент**)

**Несколько параметров:**

```
def printSred( a, b ) :  
    print ( (a + b) / 2 )
```

# Локальные и глобальные переменные

глобальная  
переменная

локальная  
переменная

```
a = 5
def qq():
    a = 1
    print(a)
qq()
print(a)
```

1

5

```
a = 5
def qq():
    print(a)
qq()
```

5

```
a = 5
def qq():
    global a
    a = 1
qq()
print(a)
```

работаем с  
глобальной  
переменной

1

# Задачи

«А»: Напишите процедуру, которая принимает параметр – натуральное число N – и выводит на экран линию из N символов '—'.

**Пример:**

Введите N:

10

-----

«В»: Напишите процедуру, которая выводит на экран в столбик все цифры переданного ей числа, начиная с первой.

**Пример:**

Введите натуральное число:

1234

1

2

3

4



# Задачи

«С»: Напишите процедуру, которая выводит на экран запись переданного ей числа в римской системе счисления.

**Пример:**

Введите натуральное число:

2013

MMXIII

# Что такое функция?

**Функция** – это вспомогательный алгоритм, который возвращает *значение-результат* (число, символ или объект другого типа).

**Задача.** Написать функцию, которая вычисляет сумму цифр числа.

**Алгоритм:**

```
сумма = 0
пока n != 0:
    сумма += n % 10
    n = n // 10
```

# Сумма цифр числа

```
def sumDigits( n ):
```

```
    sum = 0
```

```
    while n != 0:
```

```
        sum += n % 10
```

```
        n = n // 10
```

```
    return sum
```

передача  
результата

```
# основная программа
```

```
print ( sumDigits(12345) )
```

# Использование функций

```
x = 2*sumDigits(n+5)
z = sumDigits(k) + sumDigits(m)
if sumDigits(n) % 2 == 0:
    print( "Сумма цифр чётная" )
    print( "Она равна", sumDigits(n) )
```



Функция, возвращающая целое число, может использоваться везде, где и целая величина!

Одна функция вызывает другую:

```
def middle ( a, b, c ) :
    mi = min ( a, b, c )
    ma = max ( a, b, c )
    return a + b + c - mi - ma
```

ВЫЗЫВАЮТСЯ  
min И max



Что вычисляет?

# Задачи

«А»: Напишите функцию, которая находит наибольший общий делитель двух натуральных чисел.

**Пример:**

Введите два натуральных числа:

7006652 112307574

$\text{НОД}(7006652, 112307574) = 1234.$

«В»: Напишите функцию, которая определяет сумму цифр переданного ей числа.

**Пример:**

Введите натуральное число:

123

Сумма цифр числа 123 равна 6.

# Задачи

**«С»:** Напишите функцию, которая «переворачивает» число, то есть возвращает число, в котором цифры стоят в обратном порядке.

**Пример:**

Введите натуральное число:

**1234**

После переворота: 4321.

# Как вернуть несколько значений?

```
def divmod ( x, y ) :  
    d = x // y  
    m = x % y  
    return d, m
```

d – частное,  
m – остаток

```
a, b = divmod ( 7, 3 )  
print ( a, b )           # 2 1
```

```
q = divmod ( 7, 3 )  
print ( q )              # (2, 1)
```

кортеж – набор  
элементов

# Задачи

«А»: Напишите функцию, которая переставляет три переданные ей числа в порядке возрастания.

**Пример:**

Введите три натуральных числа:

10 15 5

5 10 15

«В»: Напишите функцию, которая сокращает дробь вида M/N.

**Пример:**

Введите числитель и знаменатель дроби:

25 15

После сокращения: 5/3



# Задачи

«С»: Напишите функцию, которая вычисляет наибольший общий делитель и наименьшее общее кратное двух натуральных чисел.

**Пример:**

Введите два натуральных числа:

10 15

НОД (10, 15) = 5

НОК (10, 15) = 30

# Логические функции

Задача. Найти все простые числа в диапазоне от 2 до 100.

```
for i in range(2, 1001):  
    if isPrime(i):  
        print ( i )
```

функция,  
возвращающая  
логическое значение  
(True/False)

# Функция: простое число или нет?



Какой алгоритм?

```
def isPrime ( n ) :  
    k = 2  
    while k*k <= n and n % k != 0 :  
        k += 1  
    return (k*k > n)
```

```
if k*k > n:  
    return True  
else:  
    return False
```

# Логические функции: использование



Функция, возвращающая логическое значение, может использоваться везде, где и логическая величина!

```
n = int ( input() )  
while isPrime(n):  
    print ( n, "- простое число" )  
n = int ( input() )
```

# Задачи

«А»: Напишите логическую функцию, которая определяет, является ли переданное ей число совершенным, то есть, равно ли оно сумме своих делителей, меньших его самого.

**Пример:**

Введите натуральное число:

28

Число 28 совершенное.

**Пример:**

Введите натуральное число:

29

Число 29 не совершенное.

# Задачи

**«В»:** Напишите логическую функцию, которая определяет, являются ли два переданные ей числа взаимно простыми, то есть, не имеющими общих делителей, кроме 1.

**Пример:**

Введите два натуральных числа:

**28 15**

Числа 28 и 15 взаимно простые.

**Пример:**

Введите два натуральных числа:

**28 16**

Числа 28 и 16 не взаимно простые.

# Задачи

«С»: Простое число называется гиперпростым, если любое число, получающееся из него откидыванием нескольких цифр, тоже является простым. Например, число 733 – гиперпростое, так как и оно само, и числа 73 и 7 – простые. Напишите логическую функцию, которая определяет, верно ли, что переданное ей число – гиперпростое. Используйте уже готовую функцию `isPrime`, которая приведена в учебнике.

## Пример:

Введите натуральное число:

733

Число 733 гиперпростое.

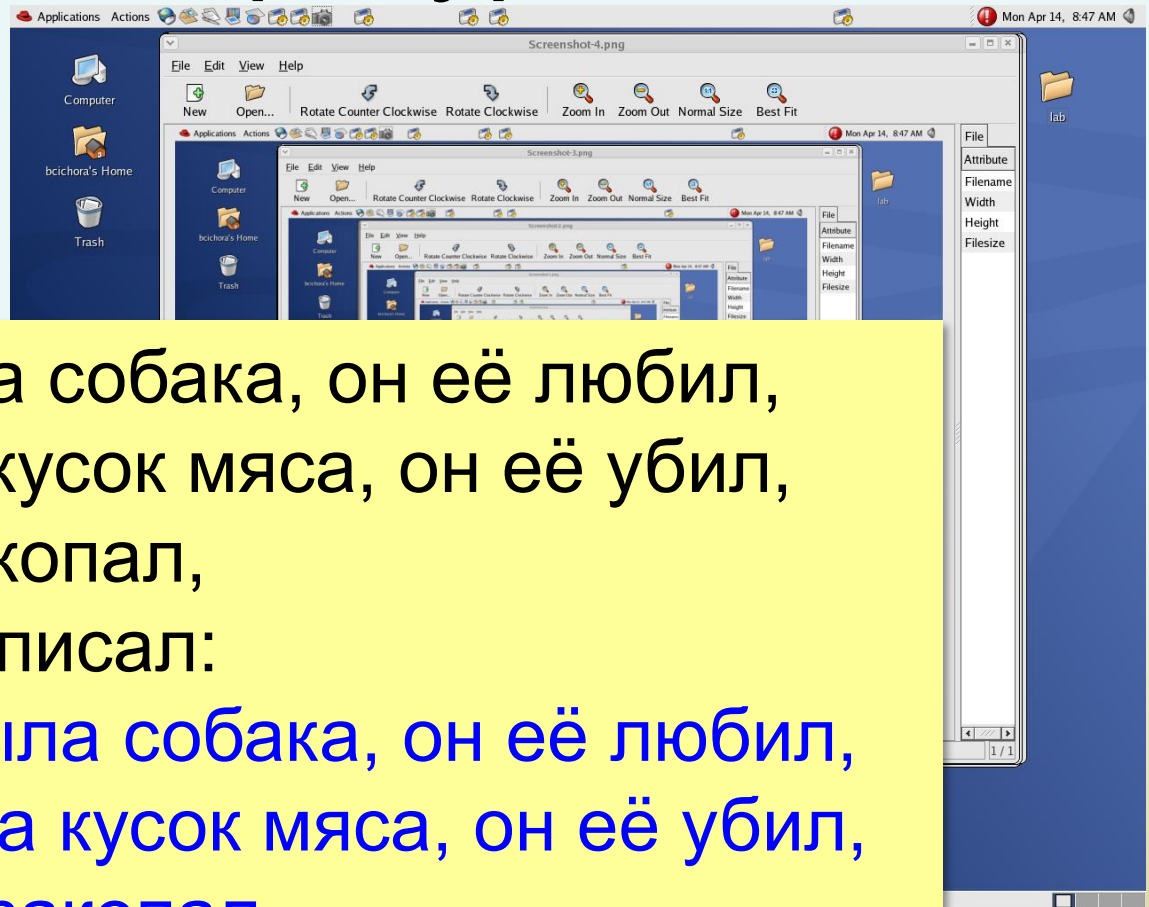
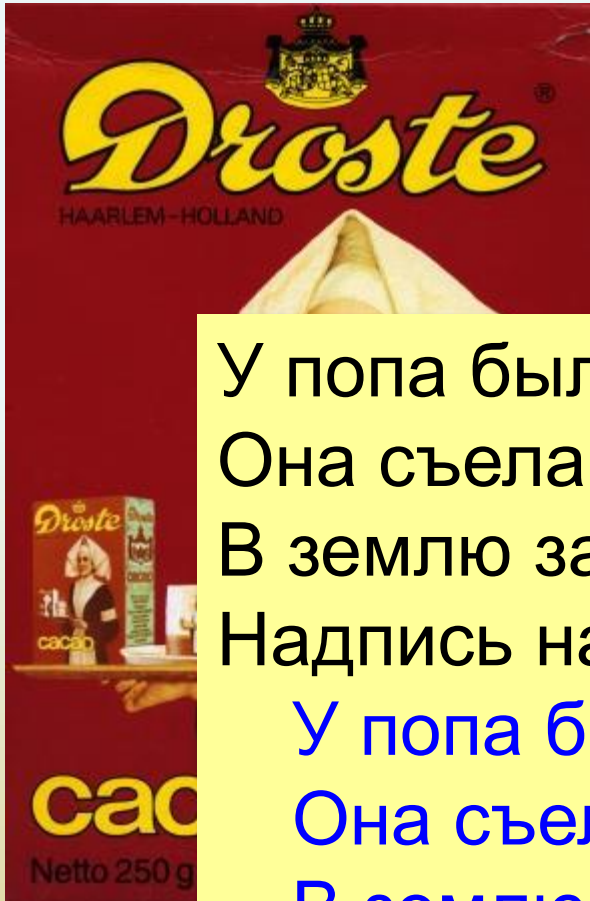
## Пример:

Введите натуральное число:

19

Число 19 не гиперпростое.

# Что такое рекурсия?



У попа была собака, он её любил,  
Она съела кусок мяса, он её убил,  
В землю закопал,  
Надпись написал:

У попа была собака, он её любил,  
Она съела кусок мяса, он её убил,  
В землю закопал,  
Надпись написал:

...



# Что такое рекурсия?

## Натуральные числа:

- 1 – натуральное число
- если  $n$  – натуральное число,  
то  $n + 1$  – натуральное число

индуктивное  
определение

**Рекурсия** — это способ определения множества объектов через само это множество на основе заданных простых базовых случаев.

## Числа Фибоначчи:

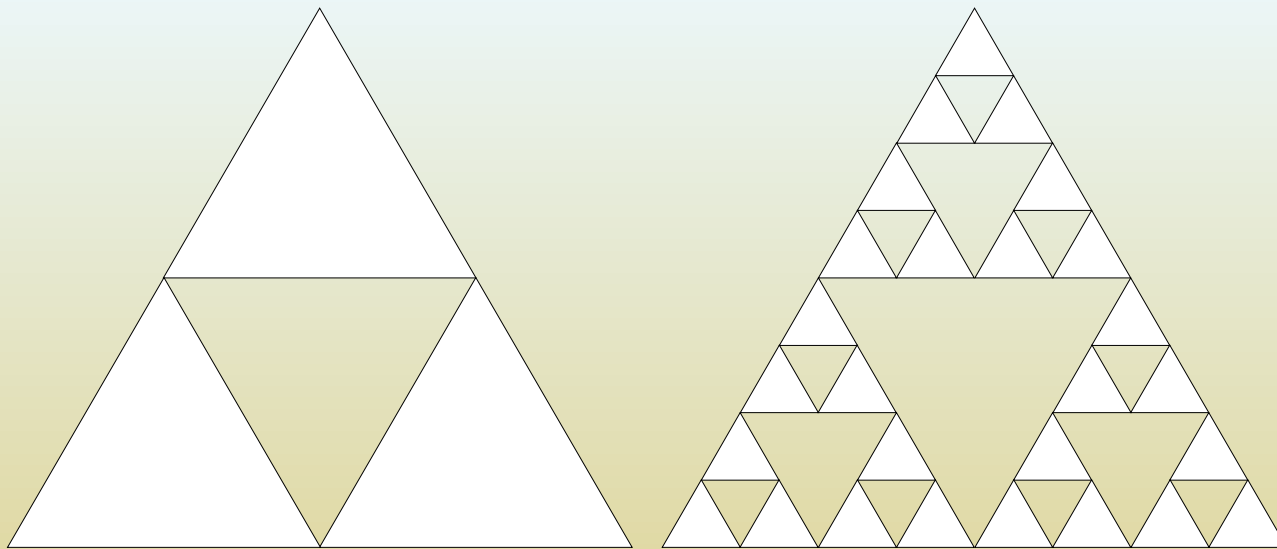
- $F_1 = F_2 = 1$
- $F_n = F_{n-1} + F_{n-2}$  при  $n > 2$

**1, 1, 2, 3, 5, 8, 13, 21, 34, ...**

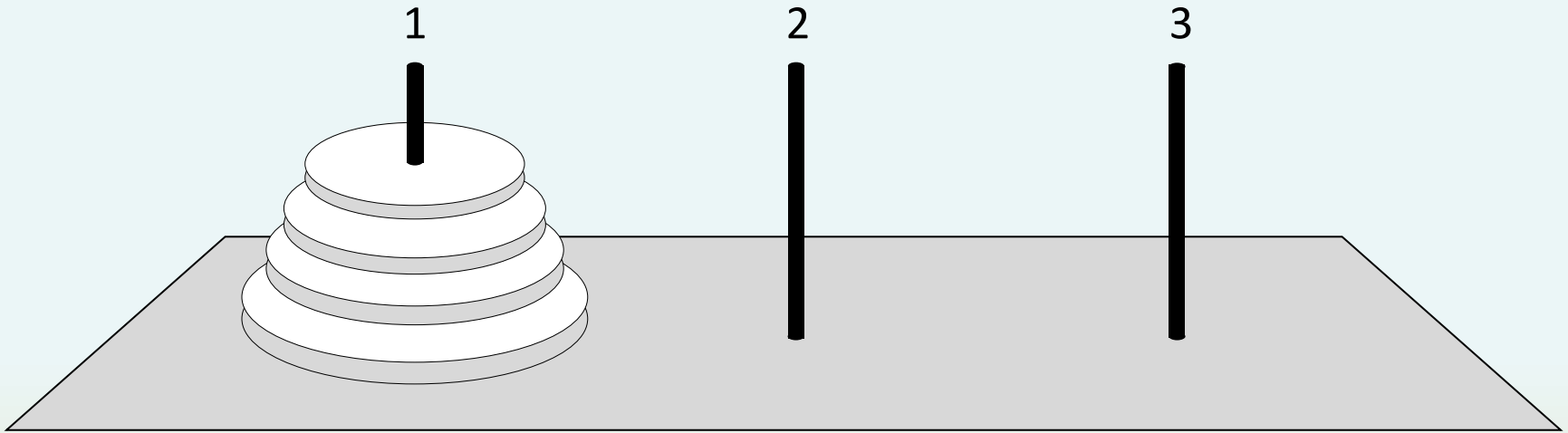
# Фракталы

**Фракталы** – геометрические фигуры, обладающие самоподобием.

**Треугольник Серпинского:**



# Ханойские башни



- за один раз переносится один диск
- класть только меньший диск на больший
- третий стержень вспомогательный

перенести (n, 1, 3)

перенести (n-1, 1, 2)

1 -> 3

перенести (n-1, 2, 3)

# Ханойские башни – процедура

сколько

откуда

куда

```
def Hanoi ( n, k, m )
```

рекурсия

```
    p = 6 - k - m
```

номер  
вспомогательного  
стержня (1+2+3=6!)

```
    Hanoi ( n-1, k, p )
```

```
    print ( k, "->", m )
```

```
    Hanoi ( n-1, p, m )
```

рекурсия

?

Что плохо?

!

Рекурсия никогда не остановится!

# Ханойские башни – процедура

Рекурсивная процедура (функция) — это процедура (функция), которая вызывает сама себя напрямую или через другие процедуры и функции.

```
def Hanoi ( n, k, m ):
```

```
    if n == 0: return
```

условие выхода из  
рекурсии

```
    p = 6 - k - m
```

```
    Hanoi ( n-1, k, p )
```

```
    print ( k, "->", m )
```

```
    Hanoi ( n-1, p, m )
```

```
# основная программа
```

```
Hanoi ( 4, 1, 3 )
```

# Вывод двоичного кода числа

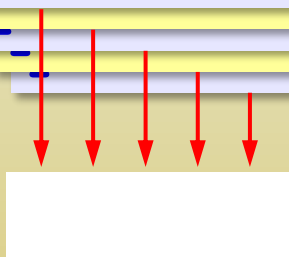
```
def printBin ( n ) :  
    if n == 0 : return  
    printBin ( n // 2 )  
    print ( n % 2 , end = "" )
```

условие выхода из  
рекурсии

напечатать все  
цифры, кроме  
последней

ВЫВЕСТИ  
последнюю цифру

```
printBin ( 0 )
```



Как без рекурсии?

# Вычисление суммы цифр числа

```
def sumDig ( n ):
```

```
    sum = n % 10
```

последняя цифра

```
    if n >= 10:
```

```
        sum += sumDig ( n // 10 )
```

```
    return sum
```

рекурсивный вызов



Где условие окончания рекурсии?

```
sumDig ( 1234 )
```

↓  
4 + sumDig ( 123 )

↓  
4 + 3 + sumDig ( 12 )

↓  
4 + 3 + 2 + sumDig ( 1 )

↓  
4 + 3 + 2 + 1

# Алгоритм Евклида

**Алгоритм Евклида.** Чтобы найти НОД двух натуральных чисел, нужно вычитать из большего числа меньшее до тех пор, пока меньшее не станет равно нулю. Тогда второе число и есть НОД исходных чисел.

```
def NOD ( a, b ) :  
    if a == 0 or b == 0 :  
        return a + b ;  
    if a > b :  
        return NOD ( a - b, b )  
    else :  
        return NOD ( a, b - a )
```

условие окончания  
рекурсии

рекурсивные вызовы



# Задачи

«А»: Напишите рекурсивную функцию, которая вычисляет НОД двух натуральных чисел, используя модифицированный алгоритм Евклида.

**Пример:**

Введите два натуральных числа:

**7006652 112307574**

$\text{НОД}(7006652, 112307574) = 1234$ .

«В»: Напишите рекурсивную функцию, которая раскладывает число на простые сомножители.

**Пример:**

Введите натуральное число:

**378**

$378 = 2 * 3 * 3 * 3 * 7$

# Задачи

«С»: Дано натуральное число  $N$ . Требуется получить и вывести на экран количество всех возможных *различных* способов представления этого числа в виде суммы натуральных чисел (то есть,  $1 + 2$  и  $2 + 1$  – это один и тот же способ разложения числа 3). Решите задачу с помощью рекурсивной процедуры.

## Пример:

**Введите натуральное число:**

**4**

**Количество разложений: 4.**

# Как работает рекурсия?

Факториал:

$$N! = \begin{cases} 1, & N = 1 \\ N \cdot (N-1)!, & N > 1 \end{cases}$$

```
def Fact(N) :  
    print ( "->", N )  
    if N <= 1: F = 1  
    else:  
        F = N * Fact ( N - 1 )  
    print ( "<-", N )  
    return F
```

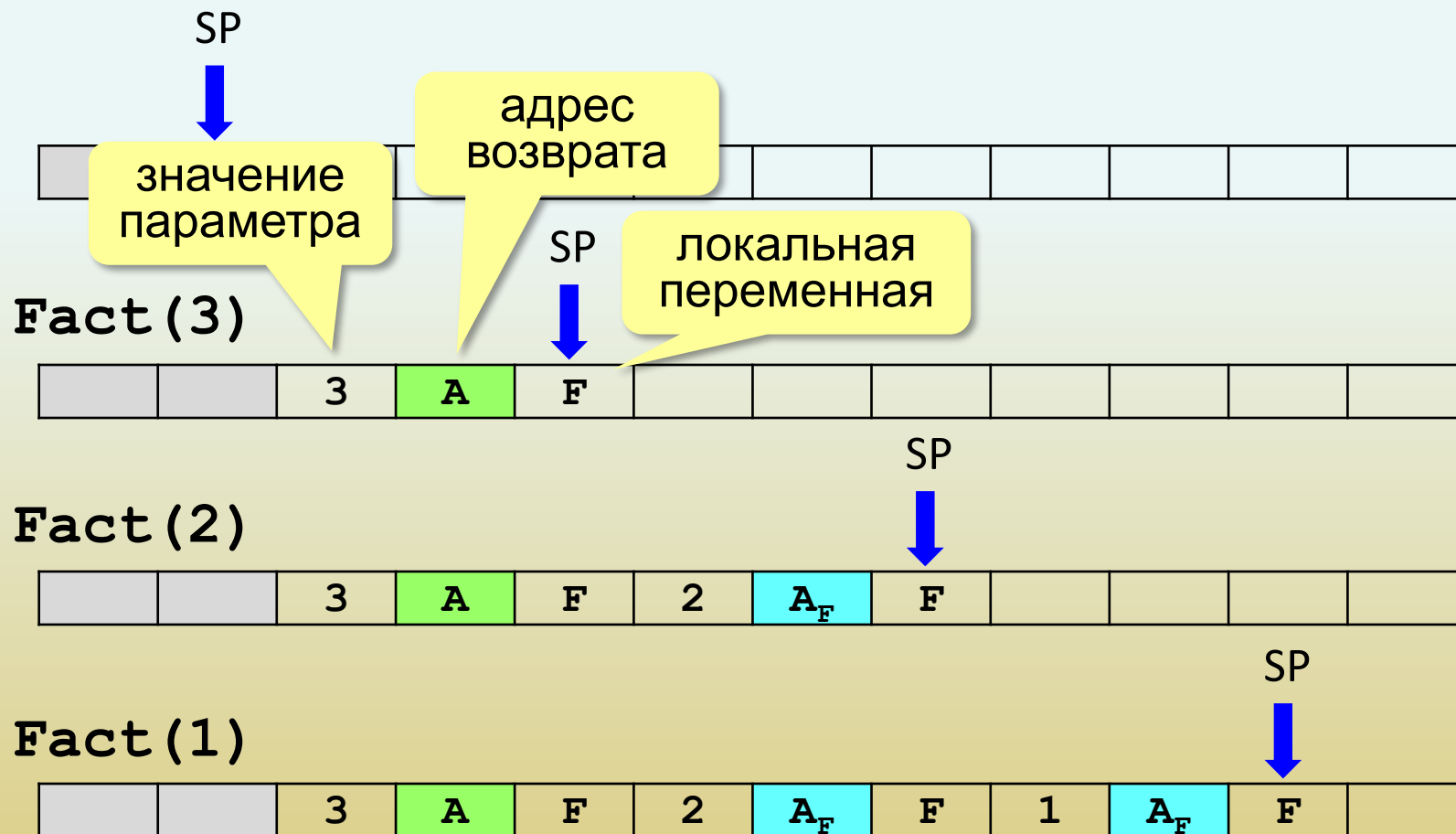
```
-> N = 3  
    -> N = 2  
        -> N = 1  
            <- N = 1  
        <- N = 2  
    <- N = 3
```



Как сохранить состояние функции перед рекурсивным вызовом?

# Стек

**Стек** – область памяти, в которой хранятся локальные переменные и адреса возврата.



# Рекурсия – «за» и «против»

- с каждым новым вызовом расходуется память в стеке (возможно переполнение стека)
- затраты на выполнение служебных операций при рекурсивном вызове



■ программа становится более короткой и понятной



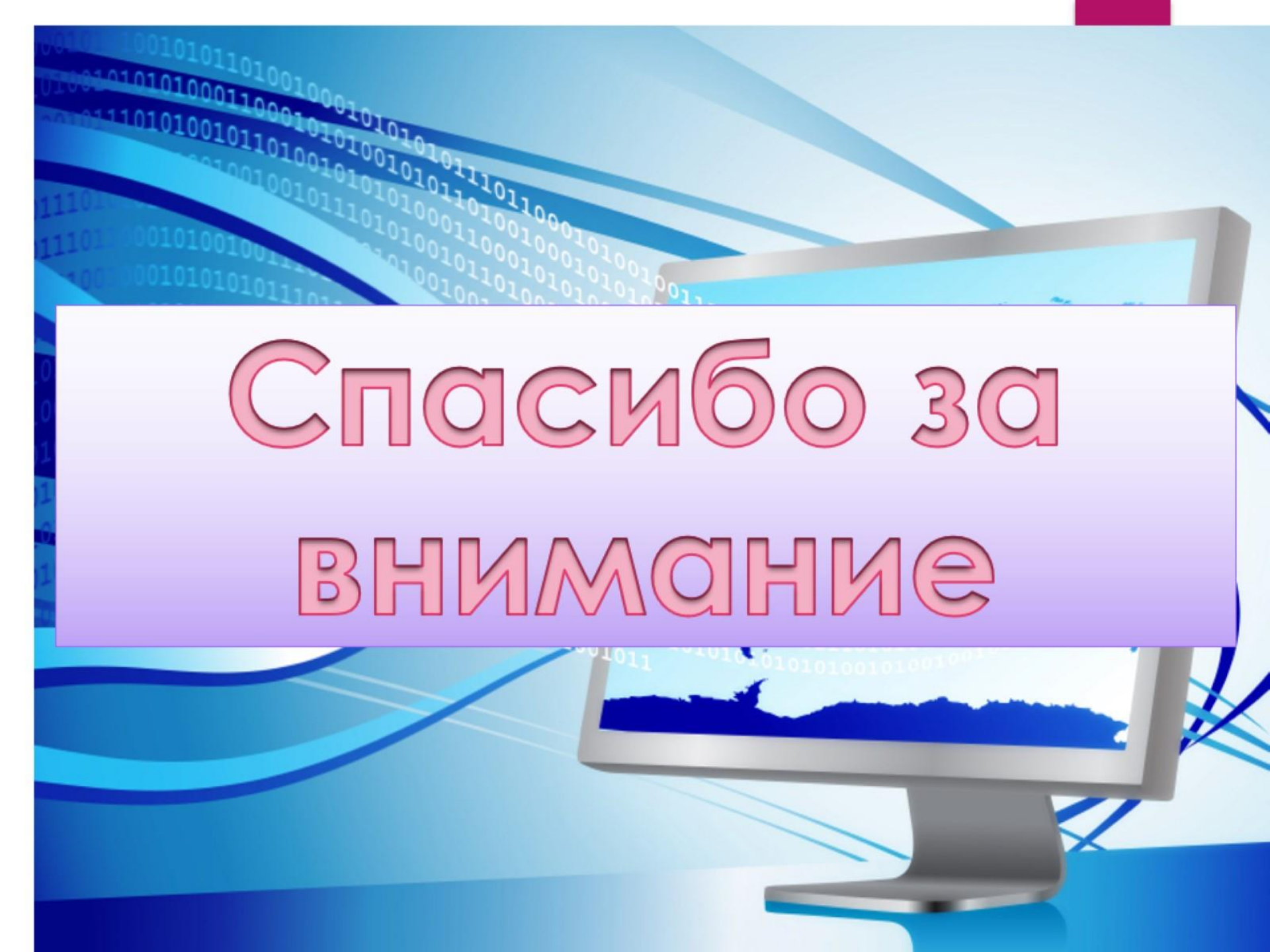
- возможно переполнение стека
- замедление работы



Любой рекурсивный алгоритм можно заменить  
итерационным!

итерационный  
алгоритм

```
def Fact ( n ) :  
    f = 1  
    for i in range ( 2 , n+1 ) :  
        f *= i  
    return f
```

The background features a blue gradient with stylized, flowing lines. Overlaid on this are several streams of binary code (0s and 1s) in a light blue color, appearing to flow from the top left towards the right. In the lower right portion of the image, there are two computer monitors. The monitor in the foreground is a flat-panel display on a stand, showing a dark blue silhouette of a landscape with a tree. The second monitor is positioned behind and slightly above the first one, showing a light blue screen. The overall aesthetic is digital and technological.

Спасибо за  
внимание